

Assign Cookies DP — Complete Learning Notes

A Comprehensive Algorithmic Journey from Paradigm to Proof

Problem Statement

Given the following inputs:

```
greed[i] = greed factor of child i
cookie[j] = size of cookie j
```

A child `i` is considered **satisfied** if and only if:

```
cookie[j] >= greed[i]
```

Constraints & Rules:

- Each child can receive at most one cookie.
- Each cookie can be assigned to at most one child.

Goal:

```
Maximize the number of satisfied children.
```

1. DP State Definition

To establish a consistent subproblem structure, we first apply an initial sorting mechanism to both input vectors:

```
sort(greed.begin(), greed.end());
sort(cookie.begin(), cookie.end());
```

State Formulation

Define `solve(i, j)` (or equivalently `dp[i][j]`) as:

"The maximum number of satisfied children obtainable by considering remaining children from index `i` onward and remaining cookies from index `j` onward."

2. Base Cases

No children left

```
if(i == n)
    return 0;
```

Reason: All children have already been processed; no further satisfaction can be achieved.

No cookies left

```
if(j == m)
    return 0;
```

Reason: No cookies remain in inventory to satisfy any remaining children.

3. DP Choices

At any given state evaluated by `solve(i, j)`, we look at the current entities:

```
child = greed[i]
cookie = cookie[j]
```

Choice 1: Take Cookie (Assign cookie `j` to child `i`)

This path is valid **only** under the structural condition:

```
cookie[j] >= greed[i]
```

Transition:

```
1 + solve(i+1, j+1)
```

Why +1? The current child becomes successfully satisfied, and both the child and cookie are consumed.

Choice 2: Skip Cookie

This branch is **always allowed**, regardless of size match.

Transition:

```
solve(i, j+1)
```

Meaning: We preserve the current child `i` but abandon the current cookie `j` to explore subsequent options.

4. Recurrence Relations

When the cookie size meets or exceeds the child's greed threshold (`cookie[j] >= greed[i]`):

```
dp[i][j] = max(
    1 + dp[i+1][j+1],
    dp[i][j+1]
)
```

When the cookie is too small to satisfy the child (`cookie[j] < greed[i]`):

```
dp[i][j] = dp[i][j+1]
```

5. Two Coding Styles

Style 1 (Explicit Recurrence)

```
if(cookie[j] >= greed[i])
{
    return max(
        1 + solve(i+1, j+1),
        solve(i, j+1)
    );
}
return solve(i, j+1);
```

Advantages: The recursive transitions are visible directly, clear to read, and seamlessly map into iterative tabulation layouts. Recommended while learning foundational DP.

Style 2 (Result Accumulation)

```
int result = 0;

if(cookie[j] >= greed[i])
{
    result = max(result, 1 + solve(i+1, j+1));
}
result = max(result, solve(i, j+1));

return result;
```

Advantages: Exceptionally clean and scalable when a state expands to contain a multitude of conditional choices. Highly prevalent across graph DP problems and general backtracking algorithms.

IMPORTANT DOUBT

Is Style 2 Wrong Because It Doesn't Explicitly Return Both Take and Skip Inside the Conditional?

No. Both structural forms are mathematically and operationally identical.

- **When a valid match exists:** `result` is initialized to the `take` branch, then compared against the `skip` branch. It resolves perfectly to `max(1 + solve(i+1, j+1), solve(i, j+1))`.
- **When a match does not exist:** The first conditional skips block execution, leaving `result = 0`, which then computes `max(0, solve(i, j+1))`. Since rewards are non-negative, this correctly evaluates to `solve(i, j+1)`.

The underlying lookup path remains completely unaffected; only the syntax of the accumulation varies.

6. Why Sorting Is Required

Sorting converts the chaotic input structures into standard monotonic sequences:

```
sort(greed.begin(), greed.end());
sort(cookie.begin(), cookie.end());
```

Post-sorting yields two core absolute properties:

1. Future children have an equal or higher greed threshold: `greed[i+1] >= greed[i]`.
2. Future cookies have an equal or larger raw size dimension: `cookie[j+1] >= cookie[j]`.

This monotonicity ensures that the indices `i` and `j` act as an accurate summary of our historical decisions, creating a valid, context-free DP state space.

IMPORTANT DOUBT

Can We Skip Sorting And Use DP Directly?

No. Consider the tracking scenario: `greed = [3, 1]`, `cookie = [1, 3]`.

The optimal matching configuration yields **2** (assigning 1 to 1, and 3 to 3).

However, an unsorted DP using index states tracking linearly returns **1**. It evaluates child `0` (greed 3) against cookie `0` (size 1) — fails, skips cookie, matches cookie `1` (size 3) with child `0`. Then child `1` (greed 1) is left with no cookies.

Without sorted monotonicity, a state vector sequence `solve(i, j)` completely forgets accessible elements bypassed earlier. Fixing this on unsorted arrays would require adding a tracking bitmask: `dp[cookieIndex][mask]`. This changes computational complexity from polynomial directly into exponential.

7. Why Does DP Explore Skip Even When Cookie Matches?

When `cookie[j] >= greed[i]`, the current cookie is large enough for the child. Yet, standard DP continues to check both choices (`Take` and `Skip`) to see which path yields the mathematical maximum.

Let's deeply analyze what the **Skip** choice actually implies:

When we evaluate `solve(i, j+1)`, we deliberately deny cookie `j` to child `i` despite it fitting perfectly, working under the hypothesis that a subsequent element (`cookie[j+1]`, `cookie[j+2]`, ...) would serve this specific child better.

However, due to our pre-sorting routine:

```
cookie[j+1] >= cookie[j], cookie[j+2] >= cookie[j], ...
```

Any future cookie is guaranteed to be at least as large as `cookie[j]`.

Now consider the alternative **Take** branch: `1 + solve(i+1, j+1)`. We fulfill the current child immediately using the smallest available valid cookie. This preserves all larger cookies for upcoming children, who are inherently greedier:

```
greed[i+1] >= greed[i], greed[i+2] >= greed[i], ...
```

Because subsequent children require larger cookie allocations to be satisfied, utilizing our smallest viable cookie right now leaves optimal inventory flexibility for future steps. Skipping a valid match would waste a small, efficient cookie and force us to expend an equal or larger cookie on this exact child later.

Thus, the **Take** path is guaranteed to be as good as or better than the **Skip** path whenever a match occurs.

The Greedy Insight

Because `Take >= Skip` is structurally guaranteed whenever `cookie[j] >= greed[i]`, the `skip` exploration branch is redundant. Eliminating this redundancy collapses the entire DP decision tree down into an elegant, linear Greedy two-pointer strategy.

8. Memoization Solution

```
int solve(int i,
          int j,
          vector<int>& greed,
          vector<int>& cookie,
          vector<vector<int>>& dp)
{
    if(i == greed.size()) return 0;
    if(j == cookie.size()) return 0;
    if(dp[i][j] != -1) return dp[i][j];

    int take = 0;
    if(cookie[j] >= greed[i])
    {
        take = 1 + solve(i+1, j+1, greed, cookie, dp);
    }

    int skip = solve(i, j+1, greed, cookie, dp);

    return dp[i][j] = max(take, skip);
}
```

9. Converting Memoization To Tabulation

We systematically translate our top-down recursive definitions into an iterative matrix configuration through 4 explicit steps:

- **Step 1 (State Mapping):** Express state transitions `solve(i, j)` directly as a matrix coordinate reference: `dp[i][j]`.
- **Step 2 (Base Conditions):** Translate base checks `if(i==n) return 0; if(j==m) return 0;` into safe initial cell values inside the matrix structure: `dp[n][*] = 0` and `dp[*][m] = 0`.
- **Step 3 (Dependency Evaluation):** Map active recursive states to explicit lookups:

↓

`solve(i+1, j+1)` maps to `dp[i+1][j+1]`

`solve(i, j+1)` maps to `dp[i][j+1]`

- **Step 4 (Loop Ordering):** Because cell `dp[i][j]` depends inherently on future lookups (namely row `i+1` and column `j+1`), the loops must iterate backwards:

```
for(int i = n - 1; i >= 0; i--)  
    for(int j = m - 1; j >= 0; j--)
```

10. Why Table Size Is $(n+1) \times (m+1)$

Critical Paradigm Doubt

The recursive tree safely evaluates boundary conditions like `solve(n, j)`, `solve(i, m)`, and `solve(n, m)`. Tabulation matrices must physically accommodate these base positions to prevent out-of-bounds segments during runtime lookups.

Thus, row `n` and column `m` must explicitly exist within our layout configuration:

```
vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
```

Visualization Grid ($n = 2, m = 2$)

	0	1	2	(j indices)
0	?	?	0	
1	?	?	0	
2	0	0	0	

(i indices)

The extra row `i == n` and extra column `j == m` store our base evaluations, letting the internal cells fill securely.

11. Tabulation Solution

```
int findContentChildren(vector<int>& greed, vector<int>& cookie) {
    int n = greed.size();
    int m = cookie.size();

    sort(greed.begin(), greed.end());
    sort(cookie.begin(), cookie.end());

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for(int i = n - 1; i >= 0; i--)
    {
        for(int j = m - 1; j >= 0; j--)
        {
            int take = 0;
            if(cookie[j] >= greed[i])
            {
                take = 1 + dp[i+1][j+1];
            }
            int skip = dp[i][j+1];

            dp[i][j] = max(take, skip);
        }
    }
    return dp[0][0];
}
```

12. Space Optimization

Observing our matrix dependencies shows that any cell `dp[i][j]` relies strictly on:

- `dp[i+1][j+1]` (The next row down)
- `dp[i][j+1]` (The current row, immediately to the right)

We do not need the full 2D table at any single time. We can reduce our memory footprint to use only two 1D rows: `next` (representing row `i+1`) and `curr` (representing row `i`).

Space Optimized Code

```
int findContentChildren(vector<int>& greed, vector<int>& cookie) {
    int n = greed.size();
    int m = cookie.size();

    sort(greed.begin(), greed.end());
    sort(cookie.begin(), cookie.end());

    vector<int> next(m + 1, 0);

    for(int i = n - 1; i >= 0; i--)
    {
        vector<int> curr(m + 1, 0);
        for(int j = m - 1; j >= 0; j--)
        {
            int take = 0;
            if(cookie[j] >= greed[i])
            {
                take = 1 + next[j+1]; // maps to dp[i+1][j+1]
            }
            int skip = curr[j+1]; // maps to dp[i][j+1]

            curr[j] = max(take, skip);
        }
        next = curr;
    }
    return next[0];
}
```

13. Final Greedy Solution

By leveraging our structural insights, we bypass DP overhead entirely to run a clean, direct two-pointer approach:

```
int findContentChildren(vector<int>& greed, vector<int>& cookie) {
    sort(greed.begin(), greed.end());
    sort(cookie.begin(), cookie.end());

    int i = 0; // Child index
    int j = 0; // Cookie index

    while(i < greed.size() && j < cookie.size())
    {
        if(cookie[j] >= greed[i])
        {
            i++; // Move to next child only when satisfied
        }
        j++; // Always consume the cookie
    }
    return i;
}
```


Complexity Summary Comparison

Algorithmic Framework	Time Complexity	Space Complexity	Implementation Overhead
Memoized DP	$O(N \times M + N \log N + M \log M)$	$O(N \times M)$ call stack/ table	High (Recursive overhead)
Tabulation DP	$O(N \times M + N \log N + M \log M)$	$O(N \times M)$ allocation	Medium (Iterative grid matrix)
Space Optimized DP	$O(N \times M + N \log N + M \log M)$	$O(M)$ array space	Medium-High (Double buffer swap)
Greedy Strategy	$O(N \log N + M \log M)$	$O(1)$ auxiliary space	Minimal (Clean two-pointer loops)

The Ultimate Lesson

This problem serves as a textbook example of algorithmic evolution. By understanding how **Recursion** → **Memoization** → **Tabulation** → **Space Optimization** → **Greedy Optimization** flows, you see how sorting elements allows a complex state-space problem to collapse directly into an elegant, optimal linear scan.